



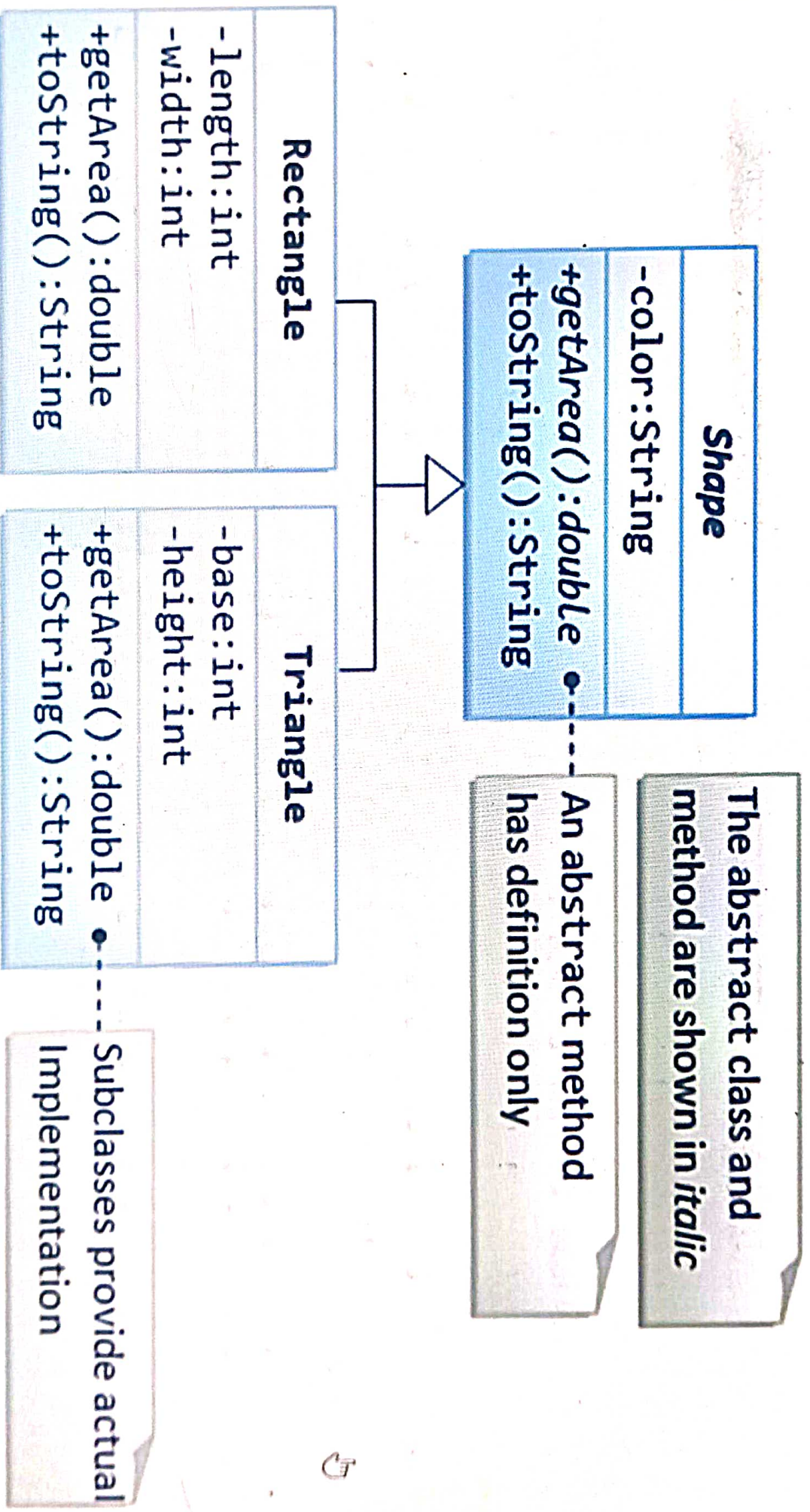
Abstract Methods and Classes

What will be covered

OBJECT

- Need of abstract method
- Abstract classes
- Access specifiers
- final keyword

- Sometimes, we require just method declaration in super-classes. This can be achieved by specifying the abstract type modifier.
- This situation may be because a class does not have complete information required for method implementation. e.g. `calcSalary()` method can not be implemented in super class `Employee` where information about entire salary structure is not available.
- But a method can not be removed totally from the class because its existence is needed for achieving runtime polymorphism.
- So a method without a method body is required in the class.
- An abstract method is a method that is declared without an implementation (without braces, and followed by a semicolon).



- If any one of the methods in the class is declared as abstract, class should be compulsorily declared as abstract.
- Class can even be declared as abstract without having any abstract method in it.
- Declaration of abstract class :

public abstract class Employee
- Abstract classes can not be instantiated but it's reference can be created.

e.g Employee e; //allowed to create reference

Employee e = new Employee(); //compilation error
- Abstract classes are incomplete in some sense (because of abstract methods), which has to be completed by the child classes.
- Subclasses need to override all the abstract methods of its super class otherwise subclass even needs to be declared as abstract

- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.
- There are four types of Java access modifiers :
 1. Default – No keyword required
 2. Private
 3. Protected
 4. Public
- Default:
 - When no access modifier is specified for a class, method, or data member, It is said to be having the default access modifier by default.
 - The data members, class or methods which are not declared using any access modifiers i.e. having default access modifier are accessible only within the same package.

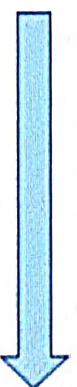
- **private :**
 - The methods or data members declared as **private** are accessible only within the class in which they are declared.
 - Top-level classes or interfaces can not be declared as **private**.
 - This is the most restricted access
- **protected :**
 - The methods or data members declared as **protected** are accessible within the same package or subclasses in different packages.
 - Protected access is majorly related to inheritance
- **public :**
 - The **public** access modifier has the widest scope among all other access modifiers.
 - Classes, methods, or data members that are declared as **public** are accessible from everywhere in the program. There is no restriction on the scope of **public** data members.

	default	private	protected	public
Same Class	Yes	Yes	Yes	Yes
Same package subclass	Yes	No	Yes	Yes
Same package non-subclass	Yes	No	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

- In Java, the final keyword can be used while declaring an entity. Using the final keyword means that the value can't be modified in the future.
- This entity can be - but is not limited to - a variable, a class or a method.
- Final keyword is exactly opposite to abstract keyword. Abstract means something missing whereas final means fully defined.

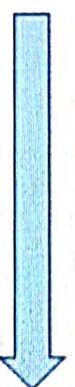


Final Variable



To create constant variables

Final Methods



Prevent Method Overriding

Final Classes



Prevent Inheritance

- When a variable is declared with final keyword, its value can't be modified, essentially, a constant.
- Note that the variable does not necessarily have to be initialized at the time of declaration. If it's declared but not yet initialized, it's called a blank final variable.
- We must initialize a final variable, otherwise compiler will throw compile-time error if it is declared as a field in class. A final variable can only be initialized once, either via an initializer or an assignment statement.
- example : `final int var = 50;`
- `var = 60` // This line would give an error
- For a final reference variable you cannot change what object it refers to. You can, however, modify the object itself

- example :

- final Reference example = new Reference(); //declaration

- example.value = 6; // Modifying the object creates no disturbance
Reference another = new Reference());

example = another; // Attempting to change the object it refers to,
creates an error

- As array and collections are even references, even though they are declared as final, object contents can be modified but it can not refer to new object altogether

- It is good practice to represent final variables in all uppercase, using underscore to separate words.

- example :

static final double PI = 3.141592653589793;

- A method, declared with the final keyword, cannot be overridden or hidden by subclasses.
- Final method can not be modified from a sub class.
- The main intention of making a method final would be that the content of the method should not be changed by any outsider.
- Examples :
 - wait(), notify(), notifyAll() methods are declared as final in class Object. So every java class has these methods but can not be overridden. These methods should be used as it is from all subclasses.

- A class declared as a final class, cannot be subclassed
- The main purpose of using a class being declared as final is to prevent the class from being subclassed.
- If a class is marked as final then no class can inherit any feature from the final class.
- Example :
 - String class in java library is declared as final. No one can create a class by using String as super class

Assignments

1. Refer Emp hierarchy created in previous assignment. Declare calSalary() method as abstract in class Emp and make necessary changes in the Emp class. Make salary data member available to subclasses of Emp
2. Refer Book hierarchy created in previous assignment. Declare calCost() method as abstract in class Book and make necessary changes in the Book class. Make price data member in Book class available to subclasses of Book.

3. Create class Account having data members as accno, name and balance. Create 2 subclasses as SavingsAccount and RecurringAccount from class Account. Recurring account will have extra data member as installment. Add void withdraw(double amt) method in class Account as abstract and override this method in subclasses. Withdrawal is not allowed from RecurringAccount and in SavingsAccount keep minimum balance as 2000 and then only remaining amount can be withdrawn. Create array of 5 Account object in main method. Accept the type of account and its related information from user. Get the amount to be withdrawn from user and call the withdraw() method.

At the end iterate the array and display all the Account objects.

4. Refer the account hierarchy - add polymorphic method double calBalance() in Account as abstract and override it in SavingAcc and RecurringAcc. Maintain int_rate as static in SavingAcc and RecurringAcc. Add the data member no_of_installments in RecurringAcc.

For SavingAcc balance to be calculated as balance with interest applicable

For RecurringAcc balance to be calculated as $\text{installment_amt} * \text{no_of_installments}$ with interest applicable.

extra :

1. Accept 2 command line arguments from the user. Parse these 2 command line arguments as int values and perform all the arithmetic operations and display the result.
2. Accept few command line arguments from the user of type int values. Create array of that many number of arguments, initialize it with values accepted from command line. Display sum and max of all array elements.
3. Compare 2 date objects for reference equality and object equality. Override equals method in class Date and check for object equality. Find out the difference.
4. Create a class Student having data members as PRN, name. Create subclasses as DACStudent and DBDASStudent. In DACStudent maintain array of strings as static data member for 6 module names and maintain array of integers (non-static) for saving marks in 6 modules.

In DBDASStudent maintain array of strings as static data member for 8 module names and maintain array of integers (non-static) for saving marks in 8 modules.

Add abstract method decideGrade() in student and override in subclasses.

4. Create a class Student having data members as PRN, name. Create subclasses as DACStudent and DBDAStudent.

In DACStudent maintain array of strings as static data member for 6 module names and maintain array of integers(non-static) for saving marks in 6 modules.

In DBDAStudent maintain array of strings as static data member for 8 module names and maintain array of integers(non-static) for saving marks in 8 modules.

Add abstract method decideGrade() in student and override in subclasses.

Add toString() method in subclasses which will display subjectwise marks, percentage and grade.

Create instance of DACStudent and DBDAStudent and call its toString() method

Assignments-1

1. Refer Emp hierarchy created in previous assignment. Declare calSalary() method as abstract in class Emp and make necessary changes in the Emp class. Make salary data member available to subclasses of Emp
2. Refer Book hierarchy created in previous assignment. Declare calCost() method as abstract in class Book and make necessary changes in the Book class. Make price data member in Book class available to subclasses of Book.

Assignments-2

1. Refer Emp hierarchy created in previous assignment. Declare calSalary() method as abstract in class Emp and make necessary changes in the Emp class. Make salary data member available to subclasses of Emp
2. Refer Book hierarchy created in previous assignment. Declare calCost() method as abstract in class Book and make necessary changes in the Book class.

3. Create a base class Account and sub classes SavAccount, CurrentAccount, RecurringAccount. Model imp state in class Account like number, name, balance etc. Save the specific state in respective derived classes like int_rate(static) in savings acc, overdraft_amt in current account, recurring_amt, no of installments and int_rate(static) in recurring account. Implement the polymorphic(override) functions like withdraw(), calBalance(), display() in all the respective classes.

In current account :
charges will be calculated based on overdraft amt.

Overdraft amt	Charges
upto 50000	- 2000
between 50000 to 100000	- 5000
between 100000 to 500000	- 8000

Assignments-2

1. Refer Emp hierarchy created in previous assignment. Declare calSalary() method as abstract in class Emp and make necessary changes in the Emp class. Make salary data member available to subclasses of Emp
 2. Refer Book hierarchy created in previous assignment. Declare calCost() method as abstract in class Book and make necessary changes in the Book class.
 3. Create a base class Account and sub classes SavAccount, CurrentAccount, RecurringAccount. Model imp state in class Account like number, name, balance etc. Save the specific state in respective derived classes like int_rate(static) in savings acc, overdraft_amt in current account, recurring_amt, no of installments and int_rate(static) in recurring account. Implement the polymorphic(override) functions like withdraw(), calBalance(), display() in all the respective classes.
- In current account :
- charges will be calculated based on overdraft amt.
- | Overdraft amt | Charges |
|--------------------------|---------|
| upto 50000. | - 2000 |
| between 50000 to 100000 | - 5000 |
| between 100000 to 500000 | - 8000 |
- Create array of 3 diff types of accounts and call their withdraw() and calBalance() method

Important Questions for: Abstract Class

Interview Questions

One scenario is there -
car has two basic function turn left and turn right.
one car is automatic
second car is semi automatic

How you implement?

What type of method you should add as Abstract

Asked all concept about abstract class and method

What is abstract class explain with example, difference between abstract class and interface